FPGA Throughput Measurement Bench

This folder contains the Verilog and SystemVerilog source files for a throughput measurement bench targeting **Vivado 2025.2** and the **Digilent Zybo Z7-10** FPGA board.

The design connects a pseudo-random data source to a convolutional encoder, feeds the encoded stream into a Viterbi decoder, and measures the decoded output throughput on hardware.

Source Files

File	Description
`throughput_bench_top.sv`	Top-level SystemVerilog module for the hardware throughput test.
`lfsr32.sv`	32-bit LFSR pseudo-random input data generator.
`convEncoder_Seq.v` and `convEncoder_Seq_*`	HLS-generated convolutional encoder RTL.
`viterbi_decoder.v` and `viterbi_decoder_*`	HLS-generated Viterbi decoder RTL.

The intended top module for implementation is:

```
throughput_bench_top
```

Target Platform

- Tool: **Xilinx Vivado 2025.2**
- Board: **Zybo Z7-10**
- FPGA family: Xilinx Zynq-7000
- Main clock used by the top module: `ap_clk`
- Reset: `ap_rst_n`, active-low synchronous reset

The top-level default clock frequency parameter is:

```
parameter int unsigned CLK_HZ = 125_000_000
```

This parameter is informational for the design comments and measurement calculation. Make sure the actual clock constraint in Vivado matches the clock you provide to `ap_clk`.

Design Overview

The data path is:

```
lfsr32 -> convEncoder_Seq -> viterbi_decoder -> throughput counter
```

1. `lfsr32` generates continuous 32-bit pseudo-random input words.
2. The input stream is sent to the convolutional encoder through an AXI-Stream style interface.
3. The encoder output stream is connected directly to the Viterbi decoder input.
4. The decoder output stream is always accepted by the test bench.
5. The top module counts how many decoded output bits are produced during a fixed measurement window.

Input Method

No external input data file or processor software is required for the hardware throughput test.

The input is generated internally by `lfsr32.sv`. The LFSR advances only when the encoder accepts an input word:

```
assign lfsr_enable = enc_s_TVALID & enc_s_TREADY;
```

The encoder input stream is driven continuously after reset:

```
assign enc_s_TVALID = ap_rst_n;
assign enc_s_TKEEP  = 4'hF;
assign enc_s_TSTRB  = 4'hF;
```

Packet framing is generated by `throughput_bench_top.sv`. By default, `enc_s_TLAST` is asserted once every 64 input words:

```
parameter int unsigned PACKET_SIZE = 64
```

To change the input packet size, update `PACKET_SIZE` in the top module parameter override or in the source file.

Throughput Measuring Technique

The measurement logic is implemented in `throughput_bench_top.sv` as a simple finite-state machine:

```
ST_WARMUP -> ST_MEASURE -> ST_DONE
```

1. Warmup

The design first waits for `INITIAL_DELAY` clock cycles. This gives the encoder and decoder pipelines time to fill before measurement starts.

Default:

```
parameter int unsigned INITIAL_DELAY = 4_096
```

2. Measurement Window

After warmup, the design measures for exactly `MEASURE_WINDOW` clock cycles.

Default:

```
parameter int unsigned MEASURE_WINDOW = 100_000_000
```

During this window:

- `cycs_q` increments every clock cycle.
- `bits_q` increments by 32 whenever the decoder produces a valid output word:

```
if (dec_TVALID & dec_TREADY)
    bits_d = bits_q + 64'd32;
```

Since `dec_TREADY` is tied high, every valid decoder output word is accepted.

3. Done

When the measurement window finishes, the FSM enters `ST_DONE`. `measurement_done` goes high and the counters freeze.

How To Measure Throughput On The Board

The important debug signals are already marked with Vivado debug attributes in `throughput_bench_top.sv`.

Use Vivado Hardware Manager with an ILA core to observe:

Signal	Meaning
`measurement_done`	Goes high when measurement is complete.
`state_q`	Measurement FSM state.
`bits_q`	Total decoded output bits counted during the measurement window.
`cycs_q`	Total clock cycles counted during the measurement window.

Signal	Meaning
`dec_TVALID`	Decoder output valid signal.
`dec_TREADY`	Decoder output ready signal.
`enc_s_TVALID`, `enc_s_TREADY`	Encoder input handshake.
`enc_m_TVALID`, `enc_m_TREADY`	Encoder-to-decoder handshake.

Vivado / ILA Flow

1. Add all RTL files in this folder to a Vivado 2025.2 project.
2. Set `throughput_bench_top` as the top module.
3. Apply the Zybo Z7-10 board clock and reset constraints.
4. Run synthesis and implementation.
5. Open the synthesized or implemented design and confirm that the `mark_debug` signals are connected to an ILA.
6. Generate the bitstream and program the Zybo Z7-10 board.
7. In Hardware Manager, arm the ILA.
8. Trigger on `measurement_done == 1'b1`.
9. Read the final frozen values of `bits_q` and `cycs_q`.

Throughput Calculation

Throughput is calculated from the ILA counter values:

```
throughput_bits_per_second = bits_q / (cycs_q / clock_frequency_hz)
```

Equivalent form:

```
throughput_bits_per_second = (bits_q * clock_frequency_hz) / cycs_q
```

For Mbps:

```
throughput_Mbps = ((bits_q * clock_frequency_hz) / cycs_q) / 1_000_000
```

If the default `MEASURE_WINDOW` is used with a 125 MHz clock, the measurement time is:

```
100,000,000 cycles / 125,000,000 Hz = 0.8 seconds
```

Example using a 125 MHz clock:

```
throughput_Mbps = (bits_q * 125,000,000) / cycs_q / 1,000,000
```

Because `cycs_q` should equal the measurement window when the FSM reaches `ST_DONE`, the default case can also be calculated as:

```
throughput_Mbps = (bits_q * 125,000,000) / 100,000,000 / 1,000,000
```

Notes

- Reset the design before each measurement run.
- Wait until `measurement_done` is high before reading `bits_q` and `cycs_q`.
- If the board clock is not 125 MHz, use the real clock frequency in the throughput equation.
- The measured throughput is decoded output throughput, not encoded input throughput.
- The LFSR is used only to keep the pipeline supplied with changing input data. It is not part of the throughput calculation.